MDL-002 — Principles

External publication draft

MDL-002 — Principles

> *A design language without principles becomes a style guide.*

Purpose

MDL-001 defines **why** Mosaic exists.

MDL-002 defines **how decisions are made**.

This specification establishes the principles that govern every future design, engineering and product decision within Mosaic.

Unlike implementation guidance, principles should remain relatively stable throughout the lifetime of the platform.

Whenever contributors disagree about a design decision, these principles should be used as the primary decision-making framework.

Relationship to MDL-001

The hierarchy of the Mosaic Design Language is intentional.

```
Vision

↓

Beliefs

↓

Principles

↓

Mental Model

↓

Interaction Model

↓

Composition Model

↓

Design System

↓

Implementation
```

Every principle defined within this specification should be directly traceable back to the philosophy established by **MDL-001 Vision**.

Principles do not exist independently.

They are practical expressions of the Mosaic vision.

Scope

This specification defines:

- Design principles
- Decision hierarchy
- Principle ownership
- Principle governance
- Principle application
- Design trade-offs

This specification intentionally does **not** define:

- Components
- Motion
- Colours
- Typography
- Materials
- Layouts
- Tokens

These belong to later MDL and MDS specifications.

Structure

This specification contains:

- 00 Document Control
- 01 What Is A Principle?
- 02 Principle Hierarchy
- 03 Context Before Prediction
- 04 Enhancement Before Persuasion
- 05 Content Leads
- 06 Movement Preserves Understanding
- 07 Every Feature Earns Its Place
- 08 The Platform Enables
- 09 Respect The User's Flow
- 10 Be A Companion
- 11 Applying Principles
- 12 Governance

- 13 ADRs
- 14 Contributor Guidance
- 15 Review Checklist
- Glossary
- References

The Seven Principles

The Mosaic Design Language is currently governed by seven core principles.

1. Context Before Prediction 2. Enhancement Before Persuasion 3. Content Leads 4. Movement Preserves Understanding 5. Every Feature Earns Its Place 6. The Platform Enables 7. Be A Companion

These principles intentionally describe behaviour rather than appearance.

A contributor should be able to apply them whether designing:

- a backend service
- a plugin
- a GraphQL schema
- a mobile client
- a television interface
- an administration page

Design Philosophy

Every principle exists to answer a simple question:

> ***When there are two good solutions, which one is more Mosaic?***

Without principles, design discussions become subjective.

With principles, contributors gain a common framework for evaluating proposals.

The objective is not agreement.

The objective is consistency.

Dependencies

```
MDL-001 Vision

↓

MDL-002 Principles

↓

MDL-003 Mental Model

↓

MDL-004 Interaction Model

↓

MDL-005 Composition Model
```

MDL-002 is required reading before implementing any future MDL or MDS specification.

Repository Structure

```
design/

■■■ mdl/

    ■■■ MDL-002 Principles/

        README.md

        00-document-control.md

        01-what-is-a-principle.md

        02-principle-hierarchy.md

        03-context-before-prediction.md

        04-enhancement-before-persuasion.md

        05-content-leads.md

        06-movement-preserves-understanding.md

        07-every-feature-earns-its-place.md

        08-the-platform-enables.md

        09-respect-the-users-flow.md

        10-be-a-companion.md

        11-applying-principles.md

        12-governance.md

        13-adrs.md

        14-contributor-guidance.md

        15-design-review-checklist.md

        glossary.md

        references.md
```

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

`00-document-control.md`

Document Control

Document Information

Property	Value
Document ID	MDL-002
Title	Mosaic Design Language — Principles
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Parent Specification	MDL-001 Vision
Repository	`/design/mdl/MDL-002 Principles/`

Purpose

MDL-002 transforms the philosophy established by **MDL-001 Vision** into practical decision-making rules.

Vision explains **why** Mosaic exists.

Principles explain **how decisions are made**.

Every future design decision should be explainable by referencing one or more principles contained within this specification.

If a proposal cannot be justified using these principles, it should be reconsidered before implementation begins.

Design Intent

A mature design language is not measured by the quality of its components.

It is measured by the consistency of decisions made by people who have never met one another.

MDL-002 exists so that two contributors working independently should naturally arrive at similar solutions because they are solving problems through the same philosophy.

The objective is not uniformity.

The objective is coherence.

Relationship to MDL

The Mosaic Design Language is intentionally hierarchical.

```
[mermaid]
flowchart TD
```

```
Vision["MDL-001 Vision"]

Vision --> Principles["MDL-002 Principles"]

Principles --> Mental["MDL-003 Mental Model"]

Mental --> Interaction["MDL-004 Interaction Model"]

Interaction --> Composition["MDL-005 Composition Model"]

Composition --> DesignSystem["MDS Specifications"]

DesignSystem --> Implementation["Product Implementation"]
```

Every specification beneath MDL-002 should inherit its decision-making framework from the principles defined here.

No specification may contradict these principles without formally amending MDL-002.

Principle Hierarchy

Not all principles possess equal authority.

The following precedence should be used whenever two principles appear to conflict.

Priority	Principle
1	Preserve the Vision
2	Reduce Friction
3	Preserve Immersion
4	Respect Current Context
5	Maintain Consistency
6	Improve Discoverability
7	Introduce New Capability

This hierarchy deliberately favours user experience over feature expansion.

How To Read This Specification

Each principle is intentionally structured using the same format.

Every chapter contains:

- Principle Statement
- Motivation
- Design Rationale
- Good Examples
- Anti-patterns
- Engineering Implications
- Review Questions

- Related ADRs
- Related Specifications

This structure allows contributors to quickly understand not only **what** the principle is, but **how** it should influence implementation.

Design Authority

The principles contained within this specification are considered normative.

Visual systems may evolve.

Implementation technologies may evolve.

The principles should evolve only when there is compelling evidence that they no longer support the product vision.

Changes require:

- Founder approval
- Design Systems approval
- Updated ADR
- Specification version increment

Intended Audience

MDL-002 should be considered required reading for:

- Product Designers
- UX Engineers
- Frontend Engineers
- Platform Engineers
- Plugin Authors
- Community Maintainers

Knowledge of this document is assumed throughout every subsequent MDL and MDS specification.

Expected Outcome

After reading MDL-002, a contributor should be capable of answering questions such as:

- Which solution is more aligned with Mosaic?
- Should this interaction exist?
- Does this feature strengthen or weaken immersion?
- Should this become part of the core platform or remain an extension?

- Does this proposal deserve to become part of the design language?

without relying upon subjective preference.

Success Criteria

MDL-002 succeeds when:

- design reviews become faster
- contributors make similar decisions independently
- discussions reference principles instead of opinions
- implementation becomes more consistent
- future specifications become easier to write

The greatest success of a principle is that contributors eventually stop consciously referring to it because it has become part of how they naturally think about Mosaic.

Review Status

Status

Draft

Dependencies

- MDL-001 Vision

Supersedes

None.

Next File

`01-what-is-a-principle.md`

What Is A Principle?

Purpose

Before defining the principles of the Mosaic Design Language, it is important to establish what a principle actually is.

Within Mosaic, a principle is **not**:

- a feature
- a requirement
- a user story
- a coding standard
- a design trend
- a visual preference

A principle is a decision-making rule.

Its purpose is to help contributors consistently choose between multiple valid solutions.

Definition

A design principle is a durable statement that guides decision-making across every layer of the product.

Unlike implementation details, principles should survive:

- technology changes
- framework changes
- platform changes
- organisational changes
- contributor changes

A well-written principle remains useful years after the software that first implemented it has been replaced.

Why Principles Exist

Every non-trivial product eventually reaches a point where multiple solutions appear equally reasonable.

Examples include:

Should information be immediately visible...

...or progressively revealed?

Should the interface become more expressive...

...or quieter?

Should plugins create UI...

...or contribute information?

Should recommendations prioritise popularity...

...or current context?

Without principles these questions become subjective.

Different contributors produce different answers.

The product slowly loses coherence.

Principles exist to prevent this.

Design Decisions

Every design decision is an optimisation.

The only question is:

> ***What are we optimising for?***

Commercial streaming platforms typically optimise for:

- engagement
- watch time
- retention
- conversion

Traditional media servers often optimise for:

- organisation
- administration
- configuration

Mosaic intentionally optimises for:

- immersion
- continuity
- understanding
- trust
- reduced cognitive effort

Every principle defined within MDL-002 exists to reinforce those optimisation targets.

Principles Are Not Rules

Rules prescribe behaviour.

Principles guide judgement.

For example:

A rule might state:

> *"Navigation must always appear on the left."*

A principle instead asks:

> *"Does this navigation reduce friction?"*

The first constrains implementation.

The second encourages thoughtful design.

Whenever possible, Mosaic prefers principles over rules.

Rules should emerge naturally from repeatedly applying principles.

Characteristics Of A Good Principle

Every principle within MDL should satisfy the following characteristics.

Durable

It should remain useful for many years.

Technology Independent

It should not depend upon:

- Go
- GraphQL
- HTMX
- WebAssembly
- SwiftUI
- React

Technologies evolve.

Principles should not.

Actionable

A contributor should be capable of making a real design decision after reading the principle.

If a principle cannot influence behaviour, it is merely philosophy.

Testable

A reviewer should be capable of asking:

> *"Does this proposal satisfy the principle?"*

If the answer cannot be evaluated, the principle requires refinement.

Broad

A principle should apply equally to:

- backend systems
- frontend applications
- mobile clients
- television interfaces
- plugins
- documentation

The implementation may differ.

The reasoning should remain consistent.

Principles Are Hierarchical

Not every principle possesses equal authority.

Some principles exist because they support more fundamental ideas.

```
[mermaid]
flowchart TD

Vision
Vision --> Beliefs
Beliefs --> Principles
Principles --> Patterns
Patterns --> Components
Components --> Implementation
```

Implementation may evolve frequently.

Principles should evolve rarely.

Vision should evolve only when absolutely necessary.

Applying A Principle

When faced with two valid solutions, contributors should ask:

```
Which option most strongly reinforces the principle?

↓

Why?

↓

Can that reasoning be explained to another contributor?

↓

Would that reasoning still make sense in five years?
```

↓

```
If yes

Proceed.
```

The purpose of principles is not simply to justify decisions.

Their purpose is to make those decisions understandable to future contributors.

Anti-pattern

The following statement is **not** a design principle.

> *"Dark mode should use #121212."*

It is:

- implementation specific
- visually prescriptive
- technology dependent

The equivalent Mosaic principle would instead be:

> *"The interface should quietly recede behind entertainment."*

One principle.

Many possible implementations.

Principle Lifecycle

Every principle should pass through the following lifecycle.

```
[mermaid]
flowchart LR

Discovery
Discovery --> Proposal
Proposal --> Review
Review --> Accepted
Accepted --> Applied
Applied --> Validated
Validated --> LivingPrinciple["Living Principle"]
```

Principles should only be retired when evidence demonstrates that they no longer support the vision established by MDL-001.

Relationship To The Remaining Specification

The remainder of MDL-002 defines the seven governing principles of Mosaic.

Each chapter should be interpreted using the framework established here.

Future contributors are encouraged to challenge implementations.

They should challenge principles only with significant evidence and formal design review.

Review Status

Status

Draft

Next File

`02-decision-hierarchy.md`

Decision Hierarchy

Purpose

Not every design decision carries equal importance.

Changing the wording of a button is fundamentally different from redefining how Mosaic understands entertainment.

Without a hierarchy, teams frequently spend disproportionate effort debating implementation details while unknowingly violating higher-level design intent.

The purpose of this chapter is to establish the order in which design decisions should be made.

Every downstream specification, implementation and review process should follow this hierarchy.

The Hierarchy

The Mosaic Design Language is intentionally layered.

```
[mermaid]
flowchart TD

Vision["Vision"]

Beliefs["Product Beliefs"]

Principles["Principles"]

Mental["Mental Model"]

Interaction["Interaction Model"]

Composition["Composition Model"]

Patterns["Patterns"]

Components["Components"]

Implementation["Implementation"]

Vision --> Beliefs
Beliefs --> Principles
Principles --> Mental
Mental --> Interaction
Interaction --> Composition
Composition --> Patterns
Patterns --> Components
Components --> Implementation
```

Higher layers explain *why*.

Lower layers explain *how*.

No lower layer may contradict a higher layer.

Layer One

Vision

Question answered:

> *Why does Mosaic exist?*

Authority:

Highest.

Changing the vision fundamentally changes the identity of Mosaic.

Examples:

- Remove friction.
- Preserve immersion.
- Become an entertainment companion.

Layer Two

Product Beliefs

Question answered:

> *What do we believe about entertainment?*

Beliefs describe the worldview from which all principles emerge.

Examples:

- Entertainment is personal.
- Context is more valuable than prediction.
- Software exists to serve entertainment.

Layer Three

Principles

Question answered:

> *How do we make decisions?*

Principles convert philosophy into repeatable decision-making tools.

Examples:

- Content Leads.
- Movement Preserves Understanding.
- Every Feature Earns Its Place.

A principle should influence hundreds of future implementation decisions.

Layer Four

Mental Model

Question answered:

> *How should users think Mosaic works?*

The mental model defines the conceptual architecture experienced by users.

Examples:

- World
- Focus
- Context
- Composition

Users should never need to understand implementation architecture.

Layer Five

Interaction Model

Question answered:

> *How does Mosaic behave?*

Interaction defines behaviour.

Examples include:

- focus changes
- composition changes
- movement
- continuity
- transitions

Interaction is independent from appearance.

Layer Six

Composition

Question answered:

> *How should information be organised?*

Composition determines:

- hierarchy
- emphasis
- relationships
- breathing space
- adaptive layouts

Composition should communicate understanding before aesthetics.

Layer Seven

Patterns

Patterns solve recurring user problems.

Examples include:

- Continuing a series
- Reading a book
- Discovering related media
- Managing downloads

Patterns combine multiple systems into coherent experiences.

Patterns should reuse existing principles rather than invent new ones.

Layer Eight

Components

Components implement patterns.

Examples include:

- Hero
- Timeline
- Progress
- Player
- Navigation

Components should remain interchangeable.

Replacing a component should not require redefining philosophy.

Layer Nine

Implementation

The final layer.

Examples include:

- HTML
- CSS
- SwiftUI
- Compose
- Flutter
- HTMX
- React

Implementation changes frequently.

It therefore possesses the lowest architectural authority.

Reading The Hierarchy

Every design discussion should begin at the highest relevant layer.

For example.

Question:

> *Should Timeline become larger?*

Wrong discussion:

```
Timeline

↓

CSS

↓

Spacing
```

Correct discussion:

```
Current Context

↓

Composition

↓

Hierarchy

↓

Timeline
```

```
↓

Component

↓

Spacing
```

The implementation should emerge naturally from the higher-level reasoning.

Resolving Conflicts

Whenever two valid proposals exist, contributors should compare them against progressively higher layers.

Example.

Proposal A:

Adds a highly requested feature.

Proposal B:

Slightly fewer features but significantly reduces friction.

Decision process.

```
Vision

↓

Reduce Friction

↓

Proposal B
```

Proposal B wins.

Not because fewer features are inherently better.

Because the Vision possesses higher authority than feature quantity.

Principle Escalation

When implementation decisions cannot be resolved locally, escalation should proceed upwards.

```
Implementation

↓

Component

↓

Pattern

↓

Composition

↓

Interaction
```

↓

```
Mental Model
```

↓

```
Principles
```

↓

```
Vision
```

The first layer capable of resolving the disagreement should make the decision.

Escalating beyond that layer is unnecessary.

This mirrors established decision-making frameworks where higher-order principles resolve lower-level implementation conflicts. [oai_citation:0‡Design Principles](https://principles.design/field-guide/?utm_source=chatgpt.com)

Decision Ownership

Layer	Primary Owner
Vision	Founder
Product Beliefs	Founder + Design
Principles	Design Systems
Mental Model	Design Systems
Interaction	Design Systems
Composition	Design Systems
Patterns	Product Design
Components	Design System Team
Implementation	Engineering

Ownership defines stewardship.

It does not prevent contributions.

Long-Term Stability

Expected lifespan of each layer.

Layer	Expected Lifetime
Vision	Decades
Beliefs	Decades
Principles	Years
Mental Model	Years
Interaction	Years
Composition	Years

Layer	Expected Lifetime
Patterns	Months to Years
Components	Months
Implementation	Weeks to Months

This hierarchy intentionally reflects the expectation that philosophy changes significantly more slowly than technology.

Architectural Decisions

ADR	Decision
ADR-001	Design decisions are hierarchical rather than independent.
ADR-002	Higher-order decisions always take precedence over lower-order implementation concerns.
ADR-003	Components implement philosophy rather than define it.
ADR-004	Engineering implementation possesses the lowest architectural authority within MDL.

Review Status

Status

Draft

Next File

`03-principle-01-context-before-prediction.md`

Principle 01 — Context Before Prediction

Principle Statement

> ***Mosaic should first understand what someone is currently doing before attempting to determine what they might do next.***

Current context is considered a more reliable foundation for design than speculative prediction.

Whenever uncertainty exists, contributors should favour present behaviour over inferred future behaviour.

Why This Principle Exists

Most entertainment platforms attempt to answer the question:

> *"What will keep this person engaged?"*

This is a valid objective for commercial services whose success depends upon maximising viewing time.

It is **not** the objective of Mosaic.

Mosaic exists to deepen enjoyment of the entertainment a person has already chosen.

Consequently, Mosaic begins with a different question.

> ***"What is this person doing right now?"***

Everything else follows from that answer.

Definition

Within MDL, **context** is defined as:

> *The user's current entertainment activity together with the information immediately surrounding that activity.*

Context is temporary.

Context evolves naturally.

Context should never permanently define the user.

Examples include:

- currently watching *Frieren*
- currently reading *The Hobbit*
- currently listening to an album
- currently browsing Studio Ghibli films

Context is not:

- favourite genre
- long-term recommendations

- demographic assumptions
- historical engagement scores

Those are historical signals.

Context describes the present.

Design Rationale

People consume entertainment in temporary worlds.

Someone watching anime today may spend tomorrow reading a novel.

Someone immersed in a television series may spend next week listening exclusively to its soundtrack.

These temporary worlds deserve respect.

By recognising current context, Mosaic reduces unnecessary decision making.

Instead of repeatedly asking:

> *"What should I do now?"*

The interface quietly supports the activity already taking place.

This produces a calmer experience with significantly less cognitive interruption.

Understanding a person's current context is a recognised design principle because people use products within specific situations, environments and goals rather than in isolation. [oai_citation:0‡GOV.UK](https://www.gov.uk/guidance/government-design-principles?utm_source=chatgpt.com)

What Context Includes

Context may include:

- current media
- current progress
- active domain
- recent interactions
- immediate relationships
- available next actions

Context intentionally excludes:

- advertising priorities
- engagement targets
- commercial promotion
- arbitrary popularity

Examples

Good

Current Context

```
Watching

Frieren
```

Mosaic presents:

- next episode countdown
- manga continuation
- soundtrack
- production information
- cast
- related novels

Every suggestion strengthens the current experience.

Good

Current Context

```
Reading

The Lord of the Rings
```

Mosaic presents:

- reading progress
- table of contents
- author's other works
- soundtrack
- film adaptations

Again, every suggestion belongs naturally within the current world.

Poor

Current Context

```
Watching

Frieren
```

Interface presents:

- Trending Horror Films
- Popular Reality Shows

- Featured Originals
- Recommended Podcasts

None of these strengthen the current activity.

They interrupt it.

Design Consequences

Applying this principle results in several observable behaviours.

The interface becomes:

- calmer
- more predictable
- less promotional
- easier to understand
- more trustworthy

Users gradually learn that Mosaic responds to what they are doing rather than attempting to redirect them elsewhere.

Engineering Implications

Future engineering systems should preserve context wherever practical.

Examples include:

- composition engine
- runtime atmosphere
- adaptive layouts
- plugin framework
- GraphQL responses

Engineering systems should ask:

> *"What information strengthens the current context?"*

before asking:

> *"What additional information could be shown?"*

Plugin Guidance

Extensions should contribute information relevant to the user's current context.

Good examples:

Anime extension

```
Episode airs tomorrow.
```

Book extension

```
Chapter progress updated.
```

Music extension

```
Live concert announced nearby.
```

The extension contributes information.

Mosaic decides whether and how that information becomes part of the current composition.

Relationship To Other Principles

This principle influences:

- Enhancement Before Persuasion
- Content Leads
- Respect the User's Flow
- Be A Companion

It is intentionally evaluated before recommendation strategies, visual hierarchy and interaction design.

Review Questions

Before approving a proposal, reviewers should ask:

- Does this strengthen the user's current context?
- Does it reduce the need for additional decisions?
- Does it respect the user's current activity?
- Does it avoid unnecessary prediction?
- Would this still make sense if popularity metrics disappeared?

If any answer is "no", the proposal should be reconsidered.

Anti-patterns

The following behaviours violate this principle.

- Promoting unrelated content while someone is immersed in another experience.
- Assuming historical preferences outweigh present behaviour.
- Replacing contextual information with trending content.
- Interrupting current entertainment to maximise engagement.

Summary

Context is the starting point for every Mosaic experience.

Prediction may become valuable later.

Context always comes first.

By respecting the user's current world, Mosaic behaves less like an algorithm and more like a trusted companion.

Related Specifications

- MDL-001 Vision
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Architectural Decisions

ADR	Decision
ADR-005	Context is the primary design input for all user experiences.
ADR-006	Current activity is considered more valuable than inferred future behaviour.
ADR-007	Recommendation systems must strengthen current context before introducing new contexts.

Review Status

Status

Draft

Next File

`04-principle-02-enhancement-before-persuasion.md`

Principle 02 — Enhancement Before Persuasion

Principle Statement

> ***Mosaic exists to deepen the entertainment experience a user has already chosen, not persuade them to choose something else.***

The role of Mosaic is to enhance.

It is never to influence for the sake of engagement.

Why This Principle Exists

Most commercial entertainment platforms have objectives that extend beyond helping people consume media.

They optimise business outcomes.

Those outcomes naturally produce behaviours such as:

- trending lists
- promoted releases
- autoplay
- endless recommendation feeds
- "continue watching something else"

These behaviours are not inherently wrong.

They simply optimise for a different objective.

Mosaic has no commercial catalogue to promote.

It therefore has no reason to compete with the user's attention.

Definition

Within MDL, **enhancement** is defined as:

> *Providing information or functionality that naturally enriches the user's current entertainment experience.*

Enhancement should feel inevitable.

It should never feel promotional.

Persuasion, by contrast, attempts to redirect the user's attention towards another experience.

Design Rationale

Entertainment is emotionally valuable.

People rarely decide:

> *"Tonight I'll consume content."*

They decide:

> *"Tonight I want to watch Frieren."*

or

> *"I'm going to finish my book."*

or

> *"I'm listening to this album again."*

Mosaic should respect that decision.

The software's responsibility begins **after** the user has chosen.

Not before.

The Difference

Enhancement asks:

- What naturally comes next?
- What belongs here?
- What makes this experience richer?
- What removes uncertainty?

Persuasion asks:

- What keeps the user engaged?
- What should replace this experience?
- What is currently popular?
- What maximises interaction?

The distinction appears small.

It fundamentally changes product behaviour.

Examples

Good

Current Activity

```
Watching

Attack on Titan
```

Mosaic displays:

- Next episode release

- Manga continuation
- Watch order
- Soundtrack
- Related films
- Production history

Every item strengthens the current experience.

Good

Current Activity

```
Reading
```

```
Dune
```

Mosaic displays:

- Reading progress
- Character guide
- Sequels
- Audiobook availability
- Film adaptations

Again, nothing attempts to replace the current activity.

Poor

Current Activity

```
Watching
```

```
Attack on Titan
```

Interface presents:

- Top 10 Netflix Originals
- Trending Horror
- Popular Comedy
- "Users also watched..."

The current experience has been abandoned.

The interface is now competing with the user's own decision.

Design Consequences

Applying this principle results in interfaces that:

- feel calmer

- build trust
- reward curiosity
- reduce decision fatigue
- encourage deeper exploration

Over time, users begin to understand that Mosaic exists to support them rather than influence them.

That trust becomes part of the product.

Engineering Implications

Future engineering systems should distinguish between:

Enhancement Information

Information that strengthens the current context.

Examples:

- release schedules
- chapter progress
- soundtrack
- author
- cast
- production studio

Persuasive Information

Information intended primarily to redirect attention.

Examples:

- global popularity
- promoted releases
- sponsored content
- engagement rankings

The core Mosaic experience should prioritise enhancement information.

Persuasive information may exist within optional community extensions but should not become part of the default experience.

Extension Guidance

Plugins should contribute information that naturally extends the current experience.

Examples.

Anime plugin

```
Next episode
```

Book plugin

```
Chapter progress
```

Music plugin

```
Live performance nearby
```

Plugins should avoid injecting unrelated promotional content simply because it is available.

The plugin framework exists to deepen experiences.

Not fragment them.

Relationship To Other Principles

This principle reinforces:

- Context Before Prediction
- Content Leads
- Be A Companion

It is intentionally evaluated before recommendation strategies and content discovery systems.

Review Questions

Reviewers should ask:

- Does this strengthen the user's current experience?
- Does this reduce unnecessary decision making?
- Does this respect the user's existing choice?
- Is this helping...

or redirecting?

- Would the experience remain valuable if engagement metrics disappeared?

If persuasion becomes the primary objective, the proposal should normally be rejected.

Anti-patterns

The following behaviours conflict with this principle.

- Trending banners replacing current context.
- Promotional homepages.
- Recommendation loops.
- Engagement-first ranking.

- Artificial urgency.
- "Because everyone else watched..."

These patterns optimise attention.

Mosaic optimises enjoyment.

Principle Summary

The user has already made the most important decision.

They have chosen what they wish to enjoy.

Mosaic should respect that decision.

Everything the platform presents afterwards should answer one question.

> ***"How can we make this experience even better?"***

Not:

> ***"How can we make them choose something else?"***

Related Specifications

- MDL-001 Vision
- MDL-003 Mental Model
- MDL-005 Composition Model
- MDS-003 Composition Engine

Architectural Decisions

ADR	Decision
ADR-008	Core Mosaic enhances rather than persuades.
ADR-009	Recommendation systems are subordinate to current context.
ADR-010	The platform should deepen user intent rather than replace it.

Review Status

Status

Draft

Next File

`05-principle-03-content-leads.md`

Principle 03 — Content Leads

Principle Statement

> ***Entertainment is always the primary experience. The interface exists to reveal it, never compete with it.***

Within Mosaic, content is the destination.

The interface is merely the path.

Whenever the interface becomes more memorable than the entertainment it presents, the design has failed.

Why This Principle Exists

People do not launch Mosaic because they enjoy user interfaces.

They launch Mosaic because they want to:

- watch a film
- continue an anime
- read a novel
- discover a soundtrack
- explore a franchise

The interface is therefore a supporting actor.

Not the protagonist.

Many design systems begin with components.

Mosaic begins with content.

The interface should emerge naturally from the needs of the entertainment rather than forcing entertainment into predefined layouts. This reflects content-first design approaches, where meaning and user needs are defined before interface structure. [oai_citation:0‡Content-first Design](https://contentfirstdesign.com/how-we-work-ux/?utm_source=chatgpt.com)

Definition

Within MDL, **content** refers to the thing the user actually values.

Examples include:

- films
- television
- anime
- books

- music
- artwork
- metadata
- relationships
- progress

Content does **not** refer to interface chrome.

Buttons are not content.

Navigation is not content.

Settings are not content.

The interface should always communicate the content rather than drawing attention to itself.

Design Rationale

Entertainment already possesses identity.

Books have covers.

Albums have artwork.

Films have cinematography.

Anime has key visuals.

The interface does not need to invent emotion.

It should allow existing emotion to dominate the experience.

Mosaic therefore intentionally separates responsibilities.

Content	Interface
Emotion	Structure
Identity	Hierarchy
Story	Navigation
Atmosphere	Consistency

Whenever those responsibilities become blurred, the experience becomes visually noisy.

Design Consequences

Applying this principle produces several observable behaviours.

Artwork becomes the primary visual focus.

Whitespace becomes valuable.

Typography becomes quieter.

Navigation becomes secondary.

Motion becomes restrained.

The interface gradually disappears as confidence increases.

Good Examples

Example 01

A user opens a television series.

The interface immediately establishes:

- current progress
- artwork
- title
- next episode

Supporting information is arranged around the series.

The artwork remains visually dominant.

Example 02

A user opens a novel.

The interface presents:

- book cover
- reading progress
- chapter position
- author

Navigation fades into the background.

Nothing distracts from the reading experience.

Example 03

A user begins playback.

As playback starts:

- interface chrome reduces
- overlays disappear
- controls become contextual

The entertainment now owns the screen.

Anti-patterns

The following behaviours violate this principle.

Promotional Interfaces

Large promotional banners competing with current content.

Decorative Interfaces

Heavy gradients.

Large glass effects.

Decorative animations.

Visual flourishes with no communicative purpose.

Dashboard Thinking

Attempting to display every possible piece of information simultaneously.

This forces users to interpret the interface rather than enjoy the entertainment.

Interface-Led Design

Designing layouts before understanding the content they must communicate.

The result is content forced into arbitrary containers.

Engineering Implications

Future engineering systems should expose information.

Not presentation.

Presentation should emerge from:

- content
- context
- composition
- relationships

This principle reinforces the long-term separation between information and interface established elsewhere within MDL.

Material Implications

Future MDS specifications should assume:

Artwork provides:

- colour
- emotion
- atmosphere

The interface provides:

- order
- rhythm
- clarity

This distinction should remain consistent regardless of future themes or material systems.

Plugin Guidance

Extensions should contribute meaningful content.

Not decorative interface.

Good plugin contribution:

```
Episode Release
```

↓

```
Tomorrow
```

Poor plugin contribution:

```
Custom animated dashboard
```

Plugins strengthen Mosaic by contributing knowledge.

The platform decides how that knowledge should be presented.

Relationship To Other Principles

Content Leads reinforces:

- Context Before Prediction
- Enhancement Before Persuasion
- Be A Companion

It also provides the philosophical foundation for:

- Material System
- Runtime Atmosphere
- Composition Engine

Review Questions

Before approving a proposal ask:

- Does this strengthen the entertainment experience?
- Is the interface supporting rather than competing?
- Would removing interface decoration improve clarity?
- Is the artwork still the strongest visual element?
- Is this communicating content or showcasing interface?

Design Litmus Test

A contributor should be able to cover the interface with their hand and still understand:

- what the user is watching
- where they are
- what comes next

If covering the content instead leaves only an attractive interface...

The interface has become too important.

Summary

Content is why Mosaic exists.

Everything else exists to support it.

The interface should become increasingly invisible as the entertainment becomes increasingly meaningful.

Related Specifications

- MDL-001 Vision
- MDL-005 Composition Model
- MDS-002 Material System

Architectural Decisions

ADR	Decision
ADR-011	Entertainment is the primary visual hierarchy.
ADR-012	Interface chrome exists only to support content.
ADR-013	Artwork is the primary source of emotion within Mosaic.

Review Status

Status

Draft

Next File

`06-principle-04-movement-preserves-understanding.md`

Principle 04 — Movement Preserves Understanding

Principle Statement

> ***Movement exists to explain change. It never exists simply because animation is possible.***

Within Mosaic, motion is considered a communication system.

Every animation should answer one question:

> ***"What just changed?"***

If movement cannot answer that question, it should not exist.

Why This Principle Exists

Motion is one of the most powerful tools available to interface designers.

It is also one of the easiest to misuse.

Many interfaces treat animation as decoration.

Buttons bounce.

Cards fade.

Pages slide.

These interactions may appear polished, but they rarely improve understanding.

Mosaic rejects decorative animation.

Motion should always strengthen the user's mental model of the interface.

Meaningful interface animation consistently emphasises continuity, orientation and explanation rather than visual spectacle. [oai_citation:0‡UIE All You Can Learn Library](https://aycl.uie.com/virtual_seminars/ux_in_motion_principles_for_creating_meaningful_animation_in_interfaces?utm_source=chatgpt.com)

Definition

Within MDL, movement is defined as:

> ***A visual explanation of changing state.***

Movement should communicate:

- cause
- effect
- relationship
- hierarchy
- continuity

Movement should never exist simply to:

- impress
- decorate
- entertain
- delay

Design Rationale

Every user action creates an expectation.

```
Action
```

↓

```
Expectation
```

↓

```
Response
```

Motion is responsible for connecting the action to the response.

Without movement the interface appears to teleport between unrelated states.

With meaningful movement the user understands:

- where something came from
- where it went
- why it moved
- how the interface changed

Understanding increases.

Cognitive effort decreases.

The Physical World

Humans instinctively understand physical movement.

Objects possess:

- position
- weight
- direction
- continuity

Interfaces should respect those expectations wherever practical.

Objects should appear to move through space rather than disappear and reappear elsewhere.

The goal is not realism.

The goal is comprehension.

Continuity

The most important property of motion is continuity.

Users should be capable of mentally following interface elements as they change.

For example:

```
Movie

↓

Selected

↓

Hero

↓

Playback
```

The movie should appear to evolve into playback.

Not vanish.

This preserves the relationship between the user's action and the resulting state.

Cause And Effect

Every visible movement should have an identifiable cause.

Examples.

Good:

```
User selects item

↓

Item expands

↓

Related information appears
```

Poor:

```
User selects item

↓

Entire interface animates

↓

Unrelated elements move

↓

Background changes
```

↓

`Sidebar pulses`

Only the first sequence strengthens understanding.

Good Examples

Example 01

Selecting a television series.

The selected artwork gently expands.

Related information reorganises around it.

Previously visible information politely moves aside.

The user understands that focus has changed.

Example 02

Closing playback.

Playback contracts.

Progress returns to the composition.

Continue Watching regains emphasis.

Nothing teleports.

The interface explains itself.

Example 03

Changing domains.

Anime

↓

Movies

The composition changes more substantially because the conceptual distance is greater.

Movement communicates this transition.

Anti-patterns

The following behaviours conflict with this principle.

Decorative Motion

Animation exists purely because it looks interesting.

No information is communicated.

Teleportation

Elements disappear from one location and instantly appear elsewhere.

The user loses spatial understanding.

Simultaneous Motion

Large numbers of unrelated elements animate together.

Hierarchy disappears.

Attention fragments.

Delayed Interaction

Animations become long enough that users wait for the interface.

Motion has become friction.

The Composition Model

Movement belongs to the composition.

Not to individual components.

Components should never animate independently without contributing to the overall understanding of the composition.

The composition owns movement.

Components participate.

This distinction will be formalised further within **MDL-004 Interaction Model** and **MDL-005 Composition Model**.

Adaptive Motion

Movement should adapt according to conceptual distance.

Small conceptual change:

```
Episode
```

↓

```
Next Episode
```

Small movement.

Medium conceptual change:

```
Frieren
```

↓

```
Fire Force
```

Moderate recomposition.

Large conceptual change:

```
Anime
```

↓

```
Books
```

Larger recomposition.

The amount of movement should reflect the amount of conceptual change.

Not the amount of visual change.

Accessibility

Motion must never become a dependency.

Users preferring reduced motion should receive the same understanding through:

- hierarchy
- layout
- emphasis
- timing

Animation enhances understanding.

It must never become the only mechanism through which understanding is communicated.

Plugin Guidance

Extensions should never define custom animation behaviour.

Instead, plugins contribute information.

The Mosaic composition engine determines:

- entry
- exit
- emphasis
- transition

This ensures that all movement remains consistent regardless of which extension introduced the information.

Review Questions

Before approving any motion design ask:

- Does this explain change?
- Can users understand where things moved?
- Does movement preserve continuity?
- Would removing this animation reduce understanding?
- Is this animation shorter than the user's patience?

If removing an animation improves the experience, that animation should not exist.

Litmus Test

The interface should never feel like it is performing.

It should feel like it is explaining itself.

Users should finish an interaction thinking:

> *"That made sense."*

Not:

> *"That animation looked nice."*

Summary

Movement is communication.

Movement is continuity.

Movement is understanding.

Everything else is decoration.

Related Specifications

- MDL-001 Vision
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-005 Motion System

Architectural Decisions

ADR	Decision
ADR-014	Motion exists to communicate state change rather than decorate the interface.
ADR-015	Conceptual distance determines movement intensity.
ADR-016	Components participate in composition movement rather than owning independent animation.

Review Status

Status

Draft

Next File

`07-principle-05-every-feature-earns-its-place.md`

Principle 05 — Every Feature Earns Its Place

Principle Statement

> ***Every feature introduced into Mosaic must justify the cognitive cost it imposes upon the user.***

Features are not valuable because they exist.

They are valuable because they improve the user's experience without weakening the product as a whole.

Every feature added to Mosaic becomes part of the user's mental model.

Every unnecessary feature therefore becomes permanent cognitive debt.

Why This Principle Exists

Software naturally accumulates functionality.

Every feature request appears reasonable when viewed in isolation.

Over time this leads to:

- additional settings
- additional navigation
- additional interaction patterns
- duplicated capabilities
- inconsistent terminology
- competing workflows

Eventually the product becomes harder to learn than the problems it originally solved.

Mosaic deliberately resists this trend.

Definition

A feature earns its place when it satisfies all of the following:

- solves a genuine user problem
- aligns with MDL
- strengthens existing systems
- reduces more friction than it introduces
- remains understandable without explanation

Failure in any one area should trigger redesign before implementation.

Design Rationale

Feature count is not a measure of product quality.

Two products may provide identical capabilities.

One requires twenty interactions.

The other requires five.

The second product has not become simpler because it removed features.

It became simpler because the features cooperate.

The objective of Mosaic is not feature reduction.

The objective is **feature coherence**.

The Cost Of Every Feature

Every new capability introduces hidden costs.

Examples include:

- documentation
- localisation
- accessibility
- testing
- maintenance
- discoverability
- user education
- design consistency

These costs exist whether or not users actively use the feature.

Consequently, the burden of proof belongs to the feature.

Not to the design language.

Decision Framework

Every proposed feature should answer the following questions.

Problem

What user problem is being solved?

Existing System

Can an existing Mosaic system already solve this?

User Value

Will users naturally benefit from this capability?

Complexity

How much complexity does the proposal introduce?

Alternatives

Was a simpler solution considered?

Long-term Maintenance

Will future contributors understand why this feature exists?

Feature Hierarchy

Not every feature belongs in the core platform.

```
[mermaid]
flowchart TD

UserProblem
UserProblem --> ExistingSystem
ExistingSystem --> CorePlatform
CorePlatform --> Extension
Extension --> Rejected
```

The preferred outcome is always:

Existing System

If an existing system cannot solve the problem, contributors should determine whether the capability belongs:

- within the core platform
- within an extension
- outside Mosaic entirely

Good Examples

Example 01

A user wants to continue reading after finishing an anime.

Proposal:

Expose manga continuation.

Reasoning:

Strengthens current context.

Deepens the current experience.

Requires no new interaction model.

Accepted.

Example 02

A user wants chapter progress.

Proposal:

Extend the existing Progress system.

Reasoning:

Existing system already communicates progression.

No new UI concept introduced.

Accepted.

Example 03

A plugin introduces a new media type.

Proposal:

Provide information through the existing composition engine.

Reasoning:

The plugin extends existing systems rather than introducing independent interface behaviour.

Accepted.

Poor Examples

Duplicate Navigation

Proposal:

Add another navigation area specifically for books.

Problem:

Existing navigation already supports domains.

Rejected.

New Dashboard

Proposal:

Create a separate dashboard for recommendations.

Problem:

Duplicates composition.

Weakens current context.

Rejected.

Special Case Components

Proposal:

Create a completely new widget because one plugin requires it.

Problem:

The existing tile framework should evolve instead.

Rejected pending redesign.

Core Platform vs Extension

A useful feature does not automatically belong within the core Mosaic experience.

Core functionality should remain intentionally focused.

Community extensions exist precisely because Mosaic is designed as a platform.

When deciding whether a feature belongs in the core platform, contributors should ask:

> *Does every user benefit from this capability?*

If the answer is no...

The proposal may still be valuable.

It may simply belong within an extension.

This philosophy helps prevent unnecessary platform growth while encouraging ecosystem innovation.
Platform design guidance consistently recommends strengthening core capabilities while enabling specialised functionality through extension points rather than expanding the platform indefinitely.
[oai_citation:0‡W3C](https://www.w3.org/TR/design-principles/?utm_source=chatgpt.com)

Design Debt

Features that fail to earn their place create design debt.

Examples include:

- duplicated concepts
- overlapping capabilities
- inconsistent interaction models
- special-case behaviour
- unnecessary configuration

Unlike technical debt, design debt directly affects every user.

It should therefore be removed whenever practical.

Relationship To Other Principles

This principle reinforces:

- Content Leads
- Context Before Prediction
- The Platform Enables
- Be A Companion

It intentionally discourages feature accumulation as a measure of product progress.

Review Questions

Before approving a feature ask:

- What problem does this solve?
- Does an existing system already solve it?
- What complexity does it introduce?
- Could this be implemented as an extension?
- Will contributors still understand why this exists five years from now?
- If this feature disappeared tomorrow, would users notice?

If the answers remain unclear, the feature has not yet earned its place.

Litmus Test

Every feature should be capable of completing the following sentence.

> ***"Mosaic is better because..."***

If the sentence cannot be completed clearly without referencing implementation details, the proposal should be reconsidered.

Summary

Features are investments.

Every investment should return more clarity than complexity.

The role of MDL is not to prevent innovation.

Its role is to ensure that innovation strengthens the product rather than gradually fragmenting it.

Related Specifications

- MDL-001 Vision
- MDL-005 Composition Model
- MDS-003 Composition Engine
- MDS-011 Extension Design Specification

Architectural Decisions

ADR	Decision
ADR-017	Every feature must justify its cognitive cost.
ADR-018	Existing systems should be extended before new systems are introduced.
ADR-019	Features benefiting specialised audiences should preferentially become extensions rather than core functionality.

Review Status

Status

Draft

Next File

`08-principle-06-the-platform-enables.md`

Principle 06 — The Platform Enables

Principle Statement

> ***The core Mosaic platform should solve universal problems. Specialised experiences should emerge through extension rather than expansion of the core.***

Mosaic is designed as a platform.

Not because plugins are fashionable.

Because no core team can anticipate every way people wish to enjoy entertainment.

The responsibility of the core platform is therefore to provide excellent foundations rather than infinite features.

Why This Principle Exists

Software inevitably grows.

Without clear architectural boundaries, every successful feature request becomes another permanent responsibility of the core application.

Eventually:

- navigation expands
- settings multiply
- interactions diverge
- maintenance slows
- innovation becomes more difficult

A platform avoids this by distinguishing between:

- universal capabilities
- specialised capabilities

Universal capabilities belong in the core.

Specialised capabilities belong at extension points.

Extensible platform architectures consistently favour a stable core with well-defined extension points, allowing innovation without increasing coupling or destabilising the platform. [oai_citation:0‡arc42 Quality Model](https://quality.arc42.org/approaches/plugin-architecture?utm_source=chatgpt.com)

Definition

Within MDL, **the platform** is defined as:

> *The collection of systems that every Mosaic experience depends upon.*

Examples include:

- authentication
- composition
- playback
- navigation
- search
- materials
- design language
- extension framework

Everything else should justify why it belongs inside the core.

The Responsibility Of The Core

The core platform owns:

- consistency
- composition
- accessibility
- interaction
- terminology
- behaviour
- quality

The core should not attempt to own every possible entertainment experience.

Instead, it provides the environment within which those experiences can exist.

The Responsibility Of Extensions

Extensions exist to provide:

- new domains
- specialised metadata
- community integrations
- experimental capabilities
- niche workflows

Extensions should extend Mosaic.

They should not become independent applications running inside Mosaic.

Core First

When evaluating a proposal, contributors should first ask:

> ***Is this solving a universal problem?***

If the answer is yes...

The capability probably belongs in the core.

Examples:

- playback
- progress
- collections
- navigation
- accessibility

Extension First

If the proposal instead answers:

> ***Only some users need this.***

The default assumption should become:

Extension.

Examples include:

- manga providers
- audiobook integrations
- torrent health
- franchise-specific metadata
- community statistics

The burden of proof lies with moving functionality into the core.

Not the extension ecosystem.

The Platform Contract

The platform promises:

- consistency
- stability
- accessibility
- composition

- behaviour

Extensions promise:

- knowledge
- capability
- integration
- specialisation

Neither should attempt to perform the other's responsibilities.

The UI Belongs To Mosaic

One of the most important architectural consequences of this principle is ownership of the interface.

Plugins should not create arbitrary interface.

Plugins should contribute capability.

The platform decides presentation.

This ensures:

- consistency
- accessibility
- device independence
- future evolution

Future MDS specifications are expected to formalise this separation through the Composition Engine and Information Model.

Example

Good

Anime extension contributes:

```
Episode Release

Tomorrow
```

Book extension contributes:

```
Chapter Progress

12 / 18
```

The platform determines:

- composition
- hierarchy
- movement

- materials
- interaction

Every extension therefore feels native.

Poor

Anime extension renders:

- custom navigation
- custom cards
- custom animations
- custom spacing
- custom typography

The platform has lost ownership of the experience.

Users now experience multiple competing design languages.

Platform Growth

Growth should occur through stronger systems.

Not larger systems.

Good platform evolution looks like:

```
Core System
↓
Extension Point
↓
Community Capability
```

Poor platform evolution looks like:

```
Feature
↓
Feature
↓
Feature
↓
Feature
↓
Settings
↓
Configuration
```

↓

`Complexity`

The objective is sustainable growth.

Not unlimited growth.

Relationship To Other Principles

This principle reinforces:

- Every Feature Earns Its Place
- Be A Companion
- Content Leads

It also provides the architectural foundation for:

- Extension Framework
- Composition Engine
- Runtime Systems
- Information Model

Review Questions

Before approving a proposal ask:

- Does every Mosaic user require this capability?
- Can the existing platform already support it?
- Would an extension provide the same value?
- Does this strengthen the platform or enlarge it?
- Is the proposal introducing a capability or a dependency?

If uncertainty remains, contributors should default towards extension rather than core implementation.

Litmus Test

A proposal belongs in the platform if removing it would fundamentally weaken Mosaic.

A proposal belongs in an extension if removing it only affects a particular audience or workflow.

The platform should remain intentionally small.

Its capabilities should remain intentionally powerful.

Summary

Platforms do not become successful because they contain every feature.

They become successful because they enable others to build features without weakening the foundation.

Mosaic should grow by strengthening its systems.

Not by endlessly expanding its core.

Related Specifications

- MDL-001 Vision
- MDL-005 Composition Model
- MDS-003 Composition Engine
- MDS-011 Extension Design Specification

Architectural Decisions

ADR	Decision
ADR-020	The core platform owns behaviour, consistency and presentation.
ADR-021	Extensions own specialised capability rather than interface.
ADR-022	Platform growth should occur through extension points before core expansion.

Review Status

Status

Draft

Next File

`09-principle-07-be-a-companion.md`

Principle 07 — Be A Companion

Principle Statement

> ***Every interaction within Mosaic should reinforce the feeling that the software is a trusted companion rather than a platform demanding attention.***

Mosaic exists to accompany the user.

Not to become the centre of the experience.

This principle is the behavioural identity of the entire platform.

Every future interaction should be evaluated against it.

Why This Principle Exists

Technology has become increasingly demanding.

Modern applications compete for attention through:

- notifications
- recommendations
- engagement loops
- badges
- autoplay
- promotions
- interruptions

These patterns are commercially successful because attention has become a measurable business metric.

Mosaic has a different objective.

The software exists to support a person's entertainment.

Not to become another source of competition for their attention.

This philosophy aligns closely with the ideas behind *Calm Technology*, where technology should inform without constantly demanding focus, moving between the centre and periphery of attention only when necessary.
[oai_citation:0‡Wikipedia](https://en.wikipedia.org/wiki/Calm_technology?utm_source=chatgpt.com)

Definition

Within MDL, a **companion** is software that:

- understands context
- quietly provides useful information

- respects attention
- remains predictable
- disappears when no longer needed

A companion is not passive.

It is attentive.

The distinction is important.

Passive software waits.

A companion prepares.

Design Rationale

Imagine sitting on a sofa watching a favourite television series.

A trusted friend sitting beside you might occasionally say:

> *"The next episode airs tomorrow."*

or

> *"Did you know this was adapted from a novel?"*

or

> *"The composer also worked on Interstellar."*

Then...

They stop talking.

They do not interrupt the episode every five minutes.

They do not recommend an unrelated reality television programme.

They do not insist you should be watching something more popular.

They simply help.

That is the behavioural model for Mosaic.

Presence

A companion should always feel present.

It should rarely feel visible.

Presence means the user trusts that useful information will be available when required.

Visibility means the interface continually competes for attention.

The objective is presence without intrusion.

Behaviour

The companion should:

- anticipate useful information
- reduce uncertainty
- minimise effort
- preserve continuity
- quietly reassure

The companion should not:

- persuade
- advertise
- distract
- compete
- dominate

Good Examples

Example 01

Current Context

```
Watching

The Good Wife
```

Mosaic quietly shows:

- progress
- remaining episodes
- next unwatched episode
- related legal dramas
- cast information

No unrelated promotion exists.

The companion strengthens the current experience.

Example 02

Current Context

```
Reading

Project Hail Mary
```

Mosaic quietly provides:

- reading progress
- chapter location
- audiobook availability
- author's previous novels

The software enriches the experience already taking place.

Example 03

Current Context

```
Music Playing
```

The interface fades into the background.

Playback continues uninterrupted.

Only essential controls remain immediately visible.

The companion has completed its task.

Poor Examples

Promotional Behaviour

```
Trending Today
```

↓

```
Featured Releases
```

↓

```
Popular This Week
```

↓

```
Sponsored Content
```

The software has become the primary experience.

The companion has become a salesperson.

Attention Seeking

Repeated:

- badges
- flashing indicators
- autoplay trailers
- unnecessary alerts

These behaviours interrupt rather than assist.

Excessive Conversation

Every interaction generates:

- tips
- recommendations
- explanations
- onboarding
- encouragement

The companion has forgotten when to remain silent.

Silence Is A Feature

One of the defining characteristics of a good companion is restraint.

Silence should be considered an intentional design decision.

Choosing **not** to interrupt is often the correct interaction.

The absence of unnecessary interface is therefore a positive outcome rather than missing functionality.

The Periphery

Good companions remain at the edge of attention until needed.

They move naturally between:

Peripheral awareness

↓

Focused assistance

↓

Peripheral awareness

This mirrors one of the core ideas of Calm Technology, where technology should remain mostly in the user's periphery and move into focus only when required.
[oai_citation:1‡Wikipedia](https://en.wikipedia.org/wiki/Calm_technology?utm_source=chatgpt.com)

Companion Across The Platform

This principle applies equally to:

- playback
- reading
- music
- search

- administration
- plugins

Administration should also behave like a companion.

Although more structured than entertainment experiences, it should remain calm, predictable and respectful of attention.

Plugin Guidance

Extensions should strengthen the companion.

They should never replace it.

Plugins contribute:

- information
- capability
- knowledge

The platform determines:

- timing
- presentation
- emphasis
- interaction

This ensures users experience a single companion rather than multiple competing personalities.

Review Questions

Before approving a proposal ask:

- Does this feel helpful?
- Does it interrupt unnecessarily?
- Would a trusted companion behave this way?
- Does this reduce uncertainty?
- Does the interface become quieter after completing its task?

If the proposal increases interaction without increasing understanding, it should be reconsidered.

Litmus Test

Imagine sitting beside the user.

Would saying this aloud feel helpful...

...or annoying?

If it feels annoying in conversation...

It will almost certainly feel annoying in software.

Summary

A companion is measured by trust.

Trust is earned through:

- consistency
- restraint
- understanding
- respect

Mosaic should quietly help people disappear into the worlds they love.

When it succeeds, users should remember the entertainment...

...not the software that helped them enjoy it.

Related Specifications

- MDL-001 Vision
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-003 Composition Engine

Architectural Decisions

ADR	Decision
ADR-023	Mosaic adopts the companion as its primary behavioural metaphor.
ADR-024	The interface should occupy attention only while providing value.
ADR-025	Silence is considered a valid interaction outcome.
ADR-026	Plugins extend the companion rather than creating independent personalities.

Review Status

Status

Draft

Next File

`10-when-principles-conflict.md`

When Principles Conflict

Purpose

Design principles are not algorithms.

They do not produce a single objectively correct answer.

Instead, they provide a framework for making consistent decisions when multiple good solutions exist.

This chapter defines how contributors should resolve situations where two or more principles appear to recommend different approaches.

Principles are valuable precisely because they help teams navigate trade-offs rather than simply describing aspirations. [oai_citation:0‡Design Principles](https://principles.design/field-guide/?utm_source=chatgpt.com)

Conflict Is Expected

Conflicting principles are not a flaw.

They are evidence that a design problem contains genuine trade-offs.

Examples include:

- Simplicity vs Discoverability
- Context vs Exploration
- Continuity vs Efficiency
- Platform Consistency vs Extension Flexibility

The objective is not to eliminate these tensions.

The objective is to resolve them consistently.

The Resolution Hierarchy

When principles conflict, contributors should evaluate them using the following order.

```
[mermaid]
flowchart TD

Vision
Vision --> User
User --> CurrentContext["Current Context"]
CurrentContext["Current Context"] --> Immersion
Immersion --> Understanding
Understanding --> Consistency
Consistency --> Capability
```

Higher levels always possess greater authority than lower levels.

If satisfying a lower-level principle weakens a higher-level one, the higher-level principle should prevail.

Step One

Return To The Vision

Every disagreement should begin by asking:

> *Which option more closely fulfils the vision established in MDL-001?*

If one proposal clearly removes more friction or preserves immersion more effectively, the discussion is complete.

No further comparison is required.

Step Two

Consider The User's Current Context

If both proposals satisfy the vision equally, ask:

> *Which proposal better supports the user's current activity?*

Current context always has priority over speculative future value.

Example.

Proposal A:

Expose ten recommendations.

Proposal B:

Expose one recommendation directly related to the current media.

Proposal B better supports the user's current context.

Step Three

Preserve Understanding

If both proposals support the current context equally, evaluate understanding.

Questions include:

- Which proposal is easier to explain?
- Which proposal requires fewer assumptions?
- Which proposal preserves the user's mental model?
- Which proposal introduces fewer surprises?

Understanding is considered more valuable than cleverness.

Step Four

Preserve Consistency

Consistency should be considered only after:

- vision
- context
- understanding

have already been satisfied.

Consistency is important because it builds trust.

Consistency is not important enough to preserve poor experiences.

A genuinely better interaction should not be rejected simply because it differs from an older implementation.

Instead, contributors should evaluate whether the existing implementation should evolve.

Step Five

Introduce Capability

New capability should normally be the final consideration.

The existence of a technically impressive feature is not sufficient justification for its inclusion.

Capability is valuable only when it strengthens:

- understanding
- immersion
- trust
- continuity

Worked Example

Scenario

A contributor proposes displaying globally trending films on the home composition.

The proposal increases discoverability.

However, it also distracts from the user's current entertainment context.

Principle Analysis

Principle	Result
Context Before Prediction	Conflicts
Enhancement Before Persuasion	Conflicts
Content Leads	Neutral
Movement Preserves Understanding	Neutral
Every Feature Earns Its Place	Unclear
Platform Enables	Neutral
Be A Companion	Conflicts

Three core principles conflict with the proposal.

The proposal should therefore be rejected or redesigned.

Another Example

Scenario

A plugin introduces audiobook progress.

Should it create a custom interface...

...or integrate with the existing Progress system?

Principle Analysis

Principle	Result
Every Feature Earns Its Place	Existing system preferred
Platform Enables	Existing system preferred
Be A Companion	Existing system preferred

The proposal naturally integrates with the existing Progress model.

No new interaction model is introduced.

Escalation

Some conflicts cannot be resolved locally.

When this occurs, contributors should escalate upwards through the MDL hierarchy.

```
[mermaid]
flowchart TD

Implementation
Implementation --> Component
Component --> Pattern
Pattern --> Composition
Composition --> Interaction
```

```
Interaction --> MentalModel["Mental Model"]
MentalModel["Mental Model"] --> Principles
Principles --> Vision
```

The first layer capable of resolving the disagreement should own the decision.

Escalating beyond that layer is unnecessary.

Irreconcilable Conflicts

Occasionally two principles may appear equally valid.

When this occurs:

1. Document the conflict. 2. Create an ADR. 3. Record the alternatives considered. 4. Perform user validation where appropriate. 5. Update the specification if required.

Philosophy should evolve deliberately.

Never accidentally.

Things That Should Never Resolve A Conflict

The following should never be used as primary decision criteria.

- Personal preference
- Visual novelty
- Existing implementation
- Technical convenience
- "We've always done it this way."
- Framework limitations

These may influence implementation.

They should never override MDL.

Review Questions

When reviewing conflicting proposals ask:

- Which proposal better fulfils the vision?
- Which proposal reduces more friction?
- Which proposal better respects the current context?
- Which proposal preserves understanding?
- Which proposal would still make sense five years from now?

If uncertainty remains after these questions, additional user research or prototyping should be preferred over assumption.

Summary

Principles are not intended to eliminate difficult decisions.

They exist to make difficult decisions consistent.

Whenever two good solutions exist:

The proposal that most strongly reinforces the Mosaic vision should always prevail.

Architectural Decisions

ADR	Decision
ADR-027	Principle conflicts are resolved using a defined hierarchy rather than subjective preference.
ADR-028	Vision always possesses higher authority than implementation.
ADR-029	User context is prioritised over discoverability when principles conflict.

Review Status

Status

Draft

Next File

`11-governance.md`

Principle Governance

Purpose

Principles only create value when they continue to influence decisions after they are written.

This chapter defines how the principles contained within MDL-002 are maintained, challenged, evolved and applied throughout the lifetime of Mosaic.

The purpose of governance is not enforcement.

The purpose of governance is preserving coherence while allowing thoughtful evolution.

Principles Are Product Infrastructure

Within Mosaic, principles are treated as product infrastructure.

Just as engineering protects:

- APIs
- schemas
- protocols
- compatibility

MDL protects:

- decision making
- experience
- behaviour
- product identity

Breaking a principle should therefore be considered a significant architectural change rather than a routine design decision.

Principle Lifecycle

Every principle follows the same lifecycle.

```
[mermaid]
flowchart LR

Discovery
Discovery --> Proposal
Proposal --> Review
Review --> Validation
Validation --> Accepted
Accepted --> Applied
Applied --> Maintained
Maintained --> Superseded
```

A principle should only enter the design language after repeated validation across multiple problems.

One successful feature is insufficient evidence.

Stability Expectations

The expected lifetime of a principle is measured in years rather than releases.

Artefact	Expected Lifetime
Components	Months
Patterns	Months to Years
Tokens	Years
Principles	Years
Vision	Decades

This intentional stability allows implementation to evolve while preserving the identity of the product.

Modifying A Principle

A principle should never change because:

- implementation became difficult
- another framework behaves differently
- a single feature requires an exception
- a contributor prefers another approach

A principle should change only when there is evidence that it no longer produces the desired user experience.

Evidence may include:

- repeated design conflicts
- user research
- accessibility findings
- architectural limitations
- long-term product evolution

Exception Process

Occasionally a proposal will appear to conflict with an existing principle.

Before introducing an exception contributors should ask:

1. Have we interpreted the principle correctly? 2. Can an existing system solve the problem? 3. Does the proposal reveal a gap elsewhere in MDL? 4. Is this actually a new pattern rather than an exception?

Exceptions should remain genuinely exceptional.

Repeated exceptions indicate the principle requires revision.

Principle Reviews

Every major design proposal should explicitly identify:

- which principles it reinforces
- which principles it weakens
- why those trade-offs are acceptable

Design reviews should reference principle identifiers rather than personal preference.

Example:

```
Supports

P-03
Content Leads

Supports

P-07
Be A Companion

Potential Conflict

P-05
Every Feature Earns Its Place
```

This creates objective discussions rather than subjective ones.

Principle Ownership

Each principle has a steward.

Stewardship exists to maintain quality.

It does not imply exclusive ownership.

Role	Responsibility
Founder	Product intent
Lead Design Systems Architect	Principle integrity
Engineering	Technical implications
Community	Feedback and evidence

Healthy principles improve through contribution.

They should never become personal property.

Design Drift

The greatest long-term risk to MDL is not incorrect implementation.

It is gradual design drift.

Design drift occurs when:

- terminology changes
- interaction models multiply
- exceptions accumulate
- similar problems receive different solutions
- contributors optimise locally rather than globally

Principles exist specifically to reduce drift.

Whenever drift is identified, contributors should first examine whether the relevant principle is being consistently applied.

Measuring Principle Health

A principle should periodically be evaluated against the following questions.

Clarity

Can new contributors understand the principle?

Applicability

Does it influence real design decisions?

Consistency

Do different contributors reach similar conclusions?

Longevity

Has the principle remained useful as Mosaic evolved?

Alignment

Does it still reinforce the Vision?

A principle failing several of these questions should enter formal review.

Deprecating Principles

Principles should rarely be removed.

When necessary they should become:

Superseded

rather than

Deleted.

Historical decisions remain valuable.

Future contributors should understand:

- why the principle existed
- why it changed
- what replaced it

Maintaining decision history is a common governance practice because it preserves architectural reasoning rather than only documenting the latest state. [oai_citation:0‡Ramotion](https://www.ramotion.com/blog/design-system-guide-chapter-2-principles-and-governance/?utm_source=chatgpt.com)

Governance Checklist

Before modifying MDL-002 ensure:

- [ ] User evidence exists.
- [ ] Alternatives were documented.
- [ ] Existing principles were considered.
- [ ] A new ADR has been written.
- [ ] Downstream specifications have been reviewed.
- [ ] Migration guidance has been prepared if required.

Summary

Principles are not static.

Neither are they fashionable.

They should evolve deliberately, slowly and with evidence.

A design language succeeds not because it never changes...

...but because every change strengthens the system instead of fragmenting it.

Architectural Decisions

ADR	Decision
ADR-030	Principles are treated as long-lived architectural assets.
ADR-031	Exceptions should trigger investigation before they trigger modification.
ADR-032	Principle changes require evidence rather than opinion.
ADR-033	Historical principle evolution must remain traceable.

Review Status

Status

Draft

Next File

`12-adrs.md`

Architectural Decision Records

Purpose

Architectural Decision Records (ADRs) preserve the reasoning behind the principles defined within MDL-002.

The principles themselves describe **how decisions should be made**.

The ADRs explain **why these principles exist**.

This distinction is important.

Future contributors should be able to understand not only the principle itself, but also the design problem that originally motivated it.

ADRs are intentionally lightweight records capturing context, decision and consequences so future teams understand why important architectural choices were made. [oai_citation:0‡GitHub](https://github.com/architecture-decision-record/architecture-decision-record?utm_source=chatgpt.com)

ADR Format

Every future ADR within Mosaic should follow the same structure.

```
[text]
ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications
```

Each ADR should describe **one** significant decision.

Never multiple.

ADR-041

Title

Establish Principles As The Primary Design Authority

Status

Accepted

Context

Without explicit principles, design discussions naturally become subjective.

Individual contributors optimise for local requirements.

The product gradually loses coherence.

Decision

Create MDL-002 as the authoritative source for all design decision making.

Consequences

Future specifications must reference MDL principles.

Personal preference alone is not sufficient justification for design decisions.

Alternatives Considered

- Team conventions
- Style guides
- Component documentation

Rejected because none explain *why* decisions should be made.

ADR-042

Title

Prioritise Context Over Behavioural Prediction

Status

Accepted

Context

Commercial entertainment platforms typically optimise future engagement.

Founder discovery established that Mosaic should optimise the current experience instead.

Decision

Current context becomes the primary input for experience design.

Consequences

Future systems should prioritise:

- current activity
- current focus
- current relationships

before considering predictive behaviour.

Alternatives Considered

Recommendation-first interfaces.

Rejected because they encourage attention redirection rather than immersion.

ADR-043

Title

Treat Motion As Communication

Status

Accepted

Context

Decorative animation frequently increases visual complexity without improving understanding.

Decision

Every animation within Mosaic must communicate meaningful state change.

Consequences

Future motion systems should optimise:

- continuity
- hierarchy
- explanation

rather than spectacle.

ADR-044

Title

Prefer Systems Over Features

Status

Accepted

Context

Feature-driven software naturally accumulates duplication and inconsistency.

Decision

Invest in reusable systems before individual capabilities.

Examples include:

- Composition Engine
- Material System

- Information Model
- Runtime Atmosphere

Consequences

Initial engineering investment increases.

Long-term complexity decreases.

Future features become cheaper to implement.

ADR-045

Title

Core Platform Owns Experience

Status

Accepted

Context

Extension ecosystems frequently fragment visual consistency by allowing arbitrary interface implementation.

Decision

The core platform owns:

- presentation
- composition
- interaction
- accessibility

Extensions contribute capability.

Not interface.

Consequences

Users experience one coherent product regardless of installed extensions.

ADR-046

Title

Adopt The Companion As The Behavioural Metaphor

Status

Accepted

Context

The founder consistently described Mosaic as:

> *"A nerdy friend who loves what I love."*

This metaphor appeared repeatedly throughout discovery workshops.

Decision

The companion becomes the primary behavioural metaphor for Mosaic.

Consequences

Future interaction design should favour:

- helpfulness
- restraint
- trust
- continuity

over:

- persuasion
- promotion
- engagement

ADR-047

Title

Every Feature Must Justify Cognitive Cost

Status

Accepted

Context

Every additional feature permanently increases product complexity.

Decision

Features are evaluated according to:

- user value
- cognitive cost
- system alignment

rather than implementation effort.

Consequences

The burden of proof belongs to new functionality.

Existing systems should be extended before introducing additional concepts.

ADR-048

Title

The Design Language Is Technology Independent

Status

Accepted

Context

Implementation technologies inevitably change.

The philosophy of the product should not.

Decision

MDL intentionally avoids dependence upon:

- programming languages
- frameworks
- rendering engines
- frontend libraries

Consequences

Future implementations may evolve without rewriting the design language.

ADR Relationships

```
[mermaid]
flowchart TD

ADR041["Principles"]

ADR042["Context"]

ADR043["Motion"]

ADR044["Systems"]

ADR045["Platform"]

ADR046["Companion"]

ADR047["Features"]

ADR048["Technology"]

ADR041 --> ADR042
ADR042 --> ADR046
ADR046 --> ADR047
ADR044 --> ADR045
ADR048 --> ADR044
ADR043 --> ADR046
```

The principles intentionally reinforce one another.

No ADR should be interpreted in isolation.

Superseding ADRs

Future ADRs should **supersede** earlier decisions rather than replacing them.

Historical reasoning remains valuable even after implementation evolves.

Every superseded ADR should include:

- replacement ADR
- date superseded
- rationale
- migration impact

Maintaining decision history preserves architectural knowledge and avoids repeatedly revisiting settled questions. [oai_citation:1‡GitHub](https://github.com/architecture-decision-record/architecture-decision-record?utm_source=chatgpt.com)

Review Status

Status

Draft

Next File

`13-contributor-guidance.md`

Contributor Guidance

Purpose

The purpose of this chapter is to help every contributor apply the Mosaic Design Language consistently.

MDL exists to produce coherent decisions across many contributors working independently.

A contributor should not need to ask:

> *"What would the original designers have done?"*

Instead they should be able to read MDL and arrive at similar conclusions through shared reasoning.

The objective of this guidance is not to limit creativity.

It is to focus creativity on solving user problems rather than inventing new interaction models.

Every Contribution Is A Design Decision

Design is not limited to visual interfaces.

Every pull request changes the user experience.

Examples include:

- GraphQL schemas
- plugin APIs
- database models
- navigation
- loading behaviour
- animation
- accessibility
- terminology

Every contributor therefore participates in the design language.

Design Before Implementation

Contributors should avoid beginning implementation immediately after identifying a problem.

Instead they should first ask:

1. What problem exists? 2. Why does it exist? 3. Which MDL principles apply? 4. Does an existing system already solve this? 5. What is the simplest solution?

Implementation should be the final stage of the process.

Not the first.

The Companion Test

Whenever uncertainty exists, imagine Mosaic as a trusted companion sitting beside the user.

Ask:

> *Would the companion behave like this?*

Examples.

Good

"You stopped halfway through this chapter."

"Would you like to continue?"

Good

"The next episode airs tomorrow."

Poor

"Trending Now!"

Poor

"People are watching this instead."

The companion helps.

It does not compete.

Build Systems

Contributors should favour extending systems over creating isolated features.

Prefer:

```
Progress System

↓

Audiobook Progress

↓

Book Progress

↓

Episode Progress
```

Instead of:

```
Book Progress Widget

Movie Progress Widget
```

```
Anime Progress Widget
```

```
Music Progress Widget
```

The first creates one evolving system.

The second creates four independent implementations.

Respect Existing Concepts

Before introducing:

- new terminology
- new interaction
- new component
- new motion
- new pattern

Contributors should determine whether an existing concept already satisfies the requirement.

Examples.

Instead of introducing:

```
Media Card
```

Can the existing Tile evolve?

Instead of introducing:

```
Recommendation Panel
```

Can the existing Composition model solve the problem?

Every unnecessary concept increases long-term cognitive complexity.

Design For The Next Contributor

A contribution is considered complete only when another contributor can understand why it exists.

Whenever possible include:

- rationale
- references
- ADR links
- specification references

Future maintainability is considered part of implementation quality.

Extension Authors

Extension authors should think in terms of capability.

Not interface.

Good contribution:

```
Information
↓
Relationships
↓
Capability
```

Poor contribution:

```
Custom HTML
↓
Custom CSS
↓
Custom Layout
```

The extension should enrich Mosaic.

It should never fragment it.

Challenge The Right Thing

Contributors are encouraged to challenge:

- implementations
- interactions
- patterns
- systems

They should challenge principles only with evidence.

Principles represent accumulated product knowledge.

Changing one should be treated as a significant architectural event.

Simplicity

Whenever two solutions appear equally valid, prefer the one that:

- introduces fewer concepts
- requires fewer explanations
- produces fewer exceptions
- strengthens existing systems

Simplicity is not measured by lines of code.

It is measured by the amount of understanding required from users.

Design Reviews

Every substantial contribution should identify:

- relevant MDL principle(s)
- relevant ADR(s)
- affected specifications

Example.

```
Supports

P-01 Context Before Prediction

P-03 Content Leads

ADR-042

ADR-046
```

This creates traceability between implementation and philosophy.

What Success Looks Like

A successful contributor should eventually stop consciously applying MDL.

Instead, the principles become part of how they naturally reason about design.

When multiple contributors independently produce experiences that feel unmistakably Mosaic...

MDL has succeeded.

Common Mistakes

Avoid:

- solving implementation before understanding the problem
- introducing special cases
- inventing new terminology unnecessarily
- creating one-off components
- exposing implementation details
- designing around frameworks rather than people

These mistakes create long-term design debt.

Final Guidance

If only one sentence from this chapter is remembered, it should be this:

> ***Build systems that quietly help people enjoy entertainment, not interfaces that ask people to admire software.***

Everything else within MDL exists to support that goal.

Review Status

Status

Draft

Next File

`14-design-review-checklist.md`

Design Review Checklist

Purpose

The Mosaic Design Language is intended to reduce subjective design discussions.

This checklist provides a repeatable review process that evaluates proposals against MDL rather than individual preference.

Every significant proposal should complete this checklist before implementation begins.

The objective is not to block change.

The objective is to ensure that change strengthens Mosaic rather than gradually fragmenting it.

Structured review checklists improve consistency, reduce omissions and help teams evaluate work against agreed standards rather than personal taste.
[oai_citation:0‡WPDean](https://wpdean.com/design-system-documentation/?utm_source=chatgpt.com)

When To Use This Checklist

This checklist should be completed for:

- New features
- New interaction patterns
- New components
- New navigation models
- New extension capabilities
- Material System changes
- Motion changes
- Composition changes
- Major refactors

Editorial documentation changes do not normally require a full review.

Review Outcome

Every review should produce one of the following outcomes.

Outcome	Meaning
Accepted	Ready for implementation
Accepted with Revisions	Minor changes required
Deferred	Requires additional research or prototyping
Rejected	Conflicts with MDL

Outcome	Meaning
Superseded	Replaced by another proposal

Every rejected proposal should reference the relevant MDL principle or ADR.

Section A

Vision

A1

Does the proposal strengthen the Vision defined by MDL-001?

- [ ] Yes
- [ ] No

A2

Does it reduce friction?

- [ ] Yes
- [ ] No

A3

Does it preserve immersion?

- [ ] Yes
- [ ] No

A4

Would the product feel more like Mosaic if this proposal became standard everywhere?

- [ ] Yes
- [ ] No

Section B

Principle Alignment

B1

Supports **Context Before Prediction**

- [ ] Yes
- [ ] No

B2

Supports **Enhancement Before Persuasion**

- [ ] Yes
- [ ] No

B3

Supports **Content Leads**

- [ ] Yes
- [ ] No

B4

Supports **Movement Preserves Understanding**

- [ ] Yes
- [ ] No

B5

Supports **Every Feature Earns Its Place**

- [ ] Yes
- [ ] No

B6

Supports **The Platform Enables**

- [ ] Yes
- [ ] No

B7

Supports **Be A Companion**

- [ ] Yes
- [ ] No

Section C

User Experience

C1

Does the proposal reduce cognitive effort?

- [ ] Yes
- [ ] No

C2

Does the proposal preserve the user's current context?

- [ ] Yes
- [ ] No

C3

Will users understand what changed?

- [ ] Yes
- [ ] No

C4

Does the proposal remove unnecessary decisions?

- [ ] Yes
- [ ] No

C5

Would this interaction still feel understandable with animations disabled?

- [ ] Yes
- [ ] No

Section D

System Design

D1

Can an existing Mosaic system solve this problem?

- [ ] Yes
- [ ] No

D2

If "No", has a new system been justified?

- [ ] Yes
- [ ] No

D3

Does the proposal introduce new terminology?

- [ ] Yes
- [ ] No

If yes:

Has existing terminology been evaluated first?

- [ ] Yes
- [ ] No

D4

Does the proposal duplicate existing capability?

- [ ] Yes
- [ ] No

D5

Does this increase long-term maintainability?

- [ ] Yes
- [ ] No

Section E

Platform

E1

Does this capability belong in the core platform?

- [ ] Yes
- [ ] No

E2

Would an extension provide a better solution?

- [ ] Yes
- [ ] No

E3

Does the proposal strengthen the extension ecosystem?

- [ ] Yes

- [ ] No

E4

Does the proposal expose implementation details?

- [ ] Yes
- [ ] No

If yes:

Redesign before implementation.

Section F

Accessibility

F1

Does the proposal maintain keyboard accessibility?

- [ ] Yes
- [ ] No

F2

Does it remain understandable using reduced motion?

- [ ] Yes
- [ ] No

F3

Does it maintain visual hierarchy?

- [ ] Yes
- [ ] No

F4

Does it improve accessibility rather than merely preserve it?

- [ ] Yes
- [ ] No

Section G

Companion Behaviour

G1

Would a trusted companion behave this way?

- [ ] Yes
- [ ] No

G2

Does this interrupt unnecessarily?

- [ ] Yes
- [ ] No

G3

Does this increase trust?

- [ ] Yes
- [ ] No

G4

Once the proposal has completed its purpose, does the interface quietly step back?

- [ ] Yes
- [ ] No

Automatic Rejection Criteria

The proposal should normally be rejected if it:

- introduces unnecessary friction
- weakens immersion
- duplicates an existing system
- competes with entertainment for attention
- introduces new terminology without justification
- exposes implementation details
- optimises engagement over understanding
- requires explanation before becoming understandable

Principle Traceability

Every proposal should explicitly reference:

```
Relevant Principles

Relevant ADRs

Affected Specifications

Migration Impact

Future Work
```

Traceability ensures future contributors understand not only what changed, but why it changed.

Final Question

Before approval, every reviewer should answer the following question.

> ***If every future feature behaved like this proposal, would Mosaic become a better entertainment companion?***

If the answer is uncertain, further iteration is recommended before implementation.

Review Record

```
[text]
Specification:

Reviewer(s):

Date:

Outcome:

Relevant Principles:

Relevant ADRs:

Follow-up Actions:

Implementation Notes:

Comments:
```

Review Status

Status

Draft

Next File

`glossary.md`

Glossary

Purpose

This glossary defines terminology introduced by **MDL-002 Principles**.

Definitions contained within this document supplement the core glossary established in **MDL-001 Vision**.

Where duplicate terms exist, the definition contained within the most specialised specification should take precedence for that specification only.

Terminology is considered part of the Mosaic Design Language.

Changing terminology changes how contributors think.

For this reason, terminology should evolve deliberately rather than organically. Design systems benefit from maintaining a shared vocabulary because consistent language leads to more consistent decision-making across design and engineering teams. [oai_citation:0‡Design System University](https://designsystem.university/glossary?utm_source=chatgpt.com)

A

Anti-pattern

A solution that appears useful in isolation but consistently produces poorer outcomes when adopted repeatedly.

Anti-patterns exist to teach contributors what **not** to build.

Every principle should document its own anti-patterns.

C

Capability

A piece of functionality provided by either the Mosaic core platform or an extension.

Capabilities describe *what* the system can do.

They intentionally avoid describing *how* that capability is presented.

Cognitive Cost

The amount of mental effort required to understand or operate part of the interface.

Every feature increases cognitive cost.

Good design ensures that the value introduced by a feature exceeds the additional cognitive effort required to use it.

Companion

The primary behavioural metaphor for Mosaic.

A companion:

- understands context
- offers useful assistance
- respects attention
- remains trustworthy
- disappears when no longer needed

The companion is **not**:

- promotional
- persuasive
- attention seeking
- intrusive

Context

The user's current entertainment activity.

Context is temporary.

Context changes naturally.

Context should never permanently define the user.

Context is considered a stronger design signal than behavioural prediction.

D

Decision Hierarchy

The ordered structure used to resolve design questions.

Within MDL:

```
Vision
↓
Beliefs
↓
Principles
↓
Mental Model
↓
```

```
Interaction
↓
Composition
↓
Implementation
```

Higher levels always possess greater authority.

Design Debt

Long-term complexity introduced through inconsistent design decisions.

Examples include:

- duplicated concepts
- competing terminology
- inconsistent interaction models
- unnecessary exceptions

Design debt should be treated with the same seriousness as technical debt.

E

Enhancement

Information or functionality that deepens the user's existing entertainment experience.

Examples include:

- release information
- chapter progress
- related works
- soundtrack information

Enhancement strengthens current context.

F

Feature

A user-facing capability.

Within MDL, a feature is **not** automatically considered valuable.

Every feature must justify:

- user value

- cognitive cost
- long-term maintenance
- consistency

before becoming part of Mosaic.

I

Immersion

The reduction of mental effort required to remain focused upon entertainment.

Immersion is considered the primary optimisation target of Mosaic.

P

Persuasion

Behaviour intended to redirect attention towards unrelated entertainment.

Examples include:

- trending content
- promoted releases
- engagement feeds
- popularity rankings

Persuasion is intentionally outside the scope of the core Mosaic experience.

Platform

The stable collection of systems that support every Mosaic experience.

Examples include:

- authentication
- playback
- composition
- navigation
- accessibility
- extension framework

The platform provides capability.

It should avoid solving highly specialised problems that naturally belong within extensions.

Principle

A durable decision-making rule.

Unlike implementation guidance, principles remain largely stable over time.

Principles explain **how** Mosaic chooses between competing solutions.

R

Review Question

A question used during design reviews to determine whether a proposal aligns with MDL.

Review questions intentionally reference principles rather than implementation details.

S

System

A reusable solution capable of supporting many future features.

Examples include:

- Composition Engine
- Progress System
- Navigation System

Systems are preferred over isolated feature implementations.

T

Technology Independence

The expectation that MDL remains valid regardless of implementation technology.

The design language should survive changes in:

- programming language
- rendering engine
- framework
- frontend architecture

Traceability

The ability to explain why a design decision exists.

Every significant implementation should be traceable through:

Implementation

↓

MDS

↓

MDL Principles

↓

MDL Vision

Traceability is considered a quality attribute of the Mosaic Design Language.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Vision, Beliefs, Companion
MDL-003 Mental Model	World, Focus, Context
MDL-004 Interaction Model	Behaviour, Movement
MDL-005 Composition Model	Composition, Hierarchy
MDS Specifications	Systems, Components, Tokens

Glossary Governance

New terminology should only be introduced when:

- an existing term cannot adequately describe a concept,
- the concept appears across multiple specifications,
- the new terminology simplifies future discussion.

Terminology should remain stable once established.

Renaming a core concept should require:

- Design Review
- ADR
- Specification update

This ensures contributors continue sharing a common language over the lifetime of the project.

Review Status

Status

Draft

Next File

```
references.md
```

References

Purpose

This document records the references, theories and external guidance that informed **MDL-002 — Principles**.

Unlike implementation specifications, MDL references are provided to explain the origin of ideas and to aid future contributors who wish to explore the reasoning in greater depth.

References should **inform** the design language.

They should never replace it.

MDL is intentionally opinionated.

Its authority comes from the philosophy established by Mosaic rather than any individual external source.

Reading Strategy

Contributors should approach references in the following order.

1. MDL Specifications 2. MDS Specifications 3. Product Design 4. Design Systems 5. Human Factors 6. Software Architecture

The Mosaic Design Language itself always remains the primary source of truth.

Internal References

MDL-001 — Vision

Defines:

- Product vision
- Product beliefs
- Design philosophy
- Governance

MDL-002 should always be interpreted as a practical extension of MDL-001 rather than an independent specification.

Future MDL Specifications

The following specifications build directly upon MDL-002.

- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

These specifications should reference principles rather than redefine them.

Future MDS Specifications

The following engineering specifications are expected to implement MDL-002.

- MDS-001 Design Token Architecture
- MDS-002 Material System
- MDS-003 Composition Engine
- MDS-004 Motion System
- MDS-005 Component Library

MDL explains **why**.

MDS explains **how**.

Product Design

Human-Centred Design

Referenced throughout MDL.

Particularly relevant concepts include:

- reducing cognitive effort
- user-centred decision making
- progressive disclosure
- contextual interaction

MDL extends these ideas through the concept of the entertainment companion.

Calm Technology

Referenced primarily by:

- Principle 07
- Movement philosophy
- Attention management

Important concepts adopted include:

- peripheral awareness
- respectful attention
- technology disappearing once its work is complete

Mosaic intentionally applies these ideas to entertainment software rather than ubiquitous computing.

Design Systems

The following systems influenced the structure of MDL rather than its philosophy.

Apple Human Interface Guidelines

Influences:

- experience-first thinking
- restraint
- hierarchy
- consistency

Rejected:

- visual identity
- material language

Material Design

Influences:

- documentation structure
- systematic thinking
- design token philosophy

Rejected:

- visual metaphor
- interaction philosophy

Fluent Design

Influences:

- adaptive interfaces
- cross-device thinking
- long-term governance

Atlassian Design System

Influences:

- documentation quality
- specification organisation
- engineering collaboration

Carbon Design System

Influences:

- governance
- enterprise documentation
- design maturity

Architecture

Architectural Decision Records

Referenced for:

- recording rationale
- preserving historical context
- long-term maintainability

The ADR process is adopted throughout MDL because it encourages decisions to remain understandable long after implementation changes.

Documentation As Code

MDL assumes documentation should be:

- version controlled
- peer reviewed
- traceable
- maintainable

Markdown is considered the canonical representation.

Generated PDFs are publication artefacts.

Product Philosophy

The following recurring themes influenced MDL.

Experience Before Interface

The interface exists to communicate experience.

It should never become the experience itself.

Systems Before Features

Reusable systems create more coherent products than isolated features.

Context Before Prediction

Current user intent provides a stronger design signal than speculative behavioural prediction.

Enhancement Before Persuasion

Software should deepen existing experiences rather than redirect attention.

Companion Over Platform

Software should behave like a trusted companion rather than an attention-seeking platform.

Design Documentation

The overall documentation structure of MDL draws inspiration from mature design systems that separate:

- philosophy
- principles
- foundations
- components
- governance

rather than treating a design system as merely a component library. [oai_citation:0‡Magic Patterns](https://www.magicpatterns.com/blog/design-system-documentation?utm_source=chatgpt.com)

Specification Conventions

The following conventions apply across all MDL documents.

Markdown is the authoritative source.

Every specification should:

- define purpose
- establish scope
- reference related specifications
- remain implementation independent where practical
- introduce ADRs where significant decisions occur

Future specifications should reuse this structure unless there is a compelling reason not to.

Future References

Future specifications are expected to introduce references relating to:

- Information Architecture

- Runtime Composition
- GraphQL UI
- Motion Design
- Accessibility
- Typography
- Material Systems
- Colour Science
- Human Perception
- Spatial Cognition

These references intentionally belong with the specifications that require them.

Living Document

This reference list should remain concise.

A reference should only be added if it has materially influenced:

- a principle
- an ADR
- a governance decision
- the structure of MDL

MDL is not intended to become an academic bibliography.

It is a practical engineering and design reference for contributors to Mosaic.

Completion

This concludes **MDL-002 — Principles**.

The next specification in the Mosaic Design Language is:

> ***MDL-003 — Mental Model***

Where MDL-001 explains **why** Mosaic exists and MDL-002 explains **how** decisions are made, MDL-003 defines **how Mosaic thinks**.

It introduces the conceptual model that both users and contributors should internalise before considering implementation details.